

CSCI4390-6390 Assign5

Assign5: Pattern Mining and Clustering

Due Date: Nov 20, before midnight (11:59:59PM)

Closed Itemset Mining

Implement the closed itemset mining algorithm 9.2 on page 251. For example, given the dataset:

```
A B D E
B C E
A B D E
A B C E
A B C D E
B C D
```

You can assume that each line has a transaction -- a set of items, separated by spaces. For minimum support of 3, the set of closed itemsets should be printed out in the format:

```
A,B,D,E - 3
A,B,E - 4
B,D - 4
B,C,E - 3
B,C - 4
B,E - 5
B - 6
Number of closed itemsets: 7
```

Show the frequent closed itemsets on the [chess.txt](#) dataset, using minimum support 3000.

Expectation Maximization Clustering

Implement the Expectation-Maximization (EM) algorithm for clustering (see Algorithm 13.3 in Chapter 13).

For initializing the clusters, select k random data points as the cluster centers. However, once you have chosen the centers, assign each point to the closest center to compute the covariance matrix and the prior probabilities of each cluster.

For practical purposes, you may want to use the [logexpsum trick](#) for expectation step, where you compute the log probabilities so that you can deal with very small probability values, otherwise, you may find that weights of a point for the clusters are zero. That is, if all probabilities are given as $\log P(C_i)$ and $\log P(x_j|C_i)$, then we have first compute $\log w'_{ij} = \log P(x_j|C_i) + \log P(C_i)$ (note: this is only the numerator, we have to normalize as given below). But, to compute the final w_{ij} , we have to use the logsumexp trick, since

$$\text{logsumexp}(\log w'_{1j}, \log w'_{2j}, \dots, \log w'_{kj}) = \log \left(\sum_{a=1}^k \exp \log w'_{aj} \right) = \log \left(\sum_{a=1}^k w'_{aj} \right)$$

And therefore,

$$w_{ij} = \exp \left(\log w'_{ij} - \text{logsumexp}(\log w'_{1j}, \dots, \log w'_{kj}) \right)$$

You can therefore use `scipy.special.logsumexp` function on log probabilities.

As another practical point, you can get an error when inverting the covariance matrix, so you should add a small ridge value λ value along the diagonal entries to make the matrix invertible. This can be considered as a regularized estimate of the covariance matrix, i.e.,

$$\Sigma + \lambda \mathbf{I}$$

Or just use `scipy.stats.multivariate_normal.logpdf` with `allow_singular` flag set to True.

Apply your algorithm on the [Iris Dataset](#), using $k = 3$ clusters. Use only the first four columns as the data points, and keep aside the last column -- the class label -- only for computing the cluster evaluation metric. Your output should comprise the following:

- The final mean for each cluster
- The final covariance matrix for each cluster
- Size of each cluster, after assigning each point to the cluster with highest posterior probability $P(c_i|x_j)$.
- The Normalized Mutual Information (NMI) score for your clustering. CSCI4390 can use `sklearn.metrics.normalized_mutual_info_score*`, but CSCI6390 must implement this based on Eq. 17.8 in the book.

Run your code several times (due to the different random initial center selections), and report the clustering with the best NMI score.

What to submit

- Submit your notebook named as **Assign5.ipynb** via submitty.
-

Policy on Academic Honesty

You are free to discuss how to tackle the assignment, but all coding must be your own. Please do not copy or modify code from anyone else, including code on the web. Any students caught violating the academic honesty principle will get an automatic F grade on the course and will be referred to the dean of students for disciplinary action. If you do use AI tools, you must document the details.

Contents © 2023 [Mohammed J. Zaki](#) - Powered by [Nikola](#)